

Artificial Intelligence and Machine Learning

Dr. Poline Xian

AGENDA

- ▶ Review
- ▶ Dimensional reduction
- ▶ KNN and SVM

Review

Review

- ▶ What are Discriminative Model and Generative Model?
- ▶ What are the key components in computer vision?

Dimensional Reduction

PRESENTATION TITLE

Dimensionality reduction ($\mathbb{R}^d \rightarrow \mathbb{R}^k$)

- ▶ Input data may have thousands or millions of dimensions
 - ▶ E.g. text data, audio feature data, video data, etc
- ▶ **Dimensionality reduction:** represent data with fewer dimensions
 - ▶ **Easier learning** - fewer parameters
 - ▶ **Visualization** - hard to visualize more than 3D or 4D
 - ▶ Discover **intrinsic dimension** of data
 - ▶ High dimensional data that is truly lower dimensional

Lower dimensional projections

- ▶ Rather than picking a subset of features, we can **create new features** that are **combinations of existing features**

Principal Component Analysis (PCA)

- ▶ Basic idea
 - ▶ Project d -dimensional data into k -dimensional space while preserving as much information as possible:
 - ▶ Example: Project space of 10000 audio signal features into 3-dimensional
 - ▶ Example: Project 3-D into 2-D.
 - ▶ Choose projection with **minimum reconstruction error**
 - ▶ PCA explained visually: <http://setosa.io/ev/principal-component-analysis/>

Process and Codes

- Compute covariance matrix.

$$\text{► } C = \text{var}(X) = \frac{\sum (X_i - \bar{X})(X_i - \bar{X})^T}{n-1}$$

- Eigenvalue decomposition on the covariance matrix.

$$\text{► } C\mathbf{v} = \lambda\mathbf{v}$$

$$\text{► } (C - \lambda I)\mathbf{v} = 0$$

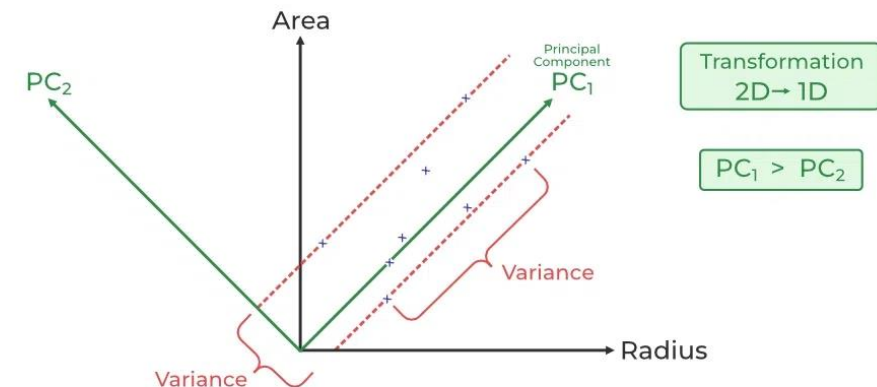
- Pick the top eigenvalues and their corresponding eigenvectors
- Data projection. Project the data onto these directions to get a low-dimensional representations: $\mathbf{y} = X\mathbf{v}$

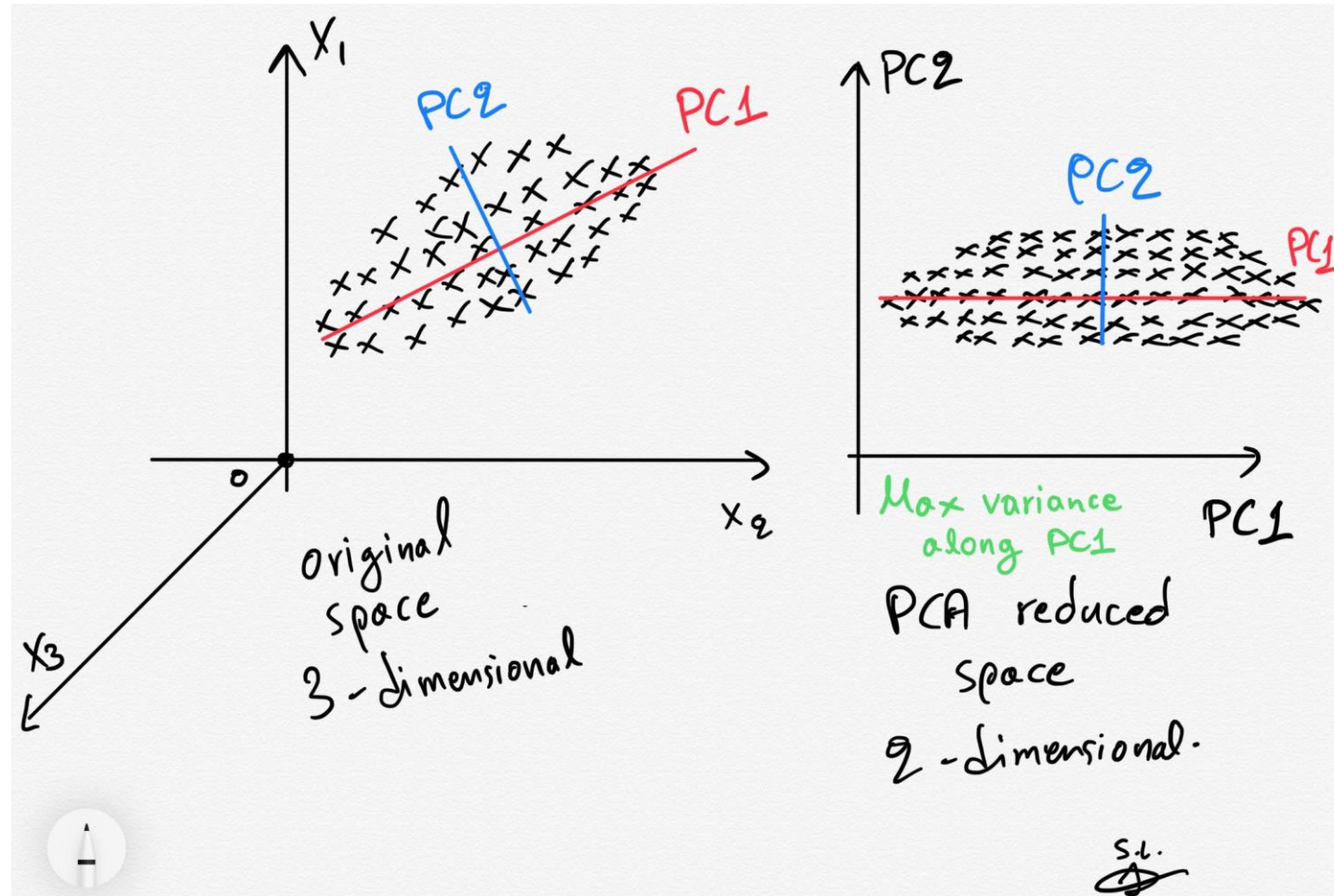
```
# generate covariance matrix
C=(X.T @ X)/(X.shape[0]-1)

#eigendecomposition
[V, PC]=np.linalg.eig(C)

# sort eigenvectors based on eigenvalues
sorted_indices=np.argsort(V)[::-1] # sort eigenvalues in descending order
sorted_PC=PC[:,sorted_indices] # sorted eigenvectors
sorted_V=V[sorted_indices] # sorted eigenvalues

# only get the first 2 principle components
PC=PC[:, :10]
X_pca=X @ PC
```





Function of PCA (Advantages)

- ▶ Noise reduction
 - ▶ Eliminates components with low variance (assumed to be noise), enhancing data clarity
- ▶ Data compression
 - ▶ Represents data with fewer components, reducing storage needs and speeding up processing.
 - ▶ High dimensional data can lead to overfitting. PCA reduces the dimensionality of the feature space and can mitigate overfitting and improve generalization.
- ▶ Computational efficiency
 - ▶ By reducing the number of features, the training process can be speeded up. A more complex model or larger datasets can be applied

Function of PCA (Disadvantages)

- ▶ Information loss
 - ▶ Reducing dimensions may lose some important information if too few components are kept.
- ▶ Assumption of linearity
 - ▶ Works best when relationships between variables are linear, and may struggle with non-linear data.
- ▶ Interpretation challenges
 - ▶ The new components are combinations of original variables, which can be hard to explain.

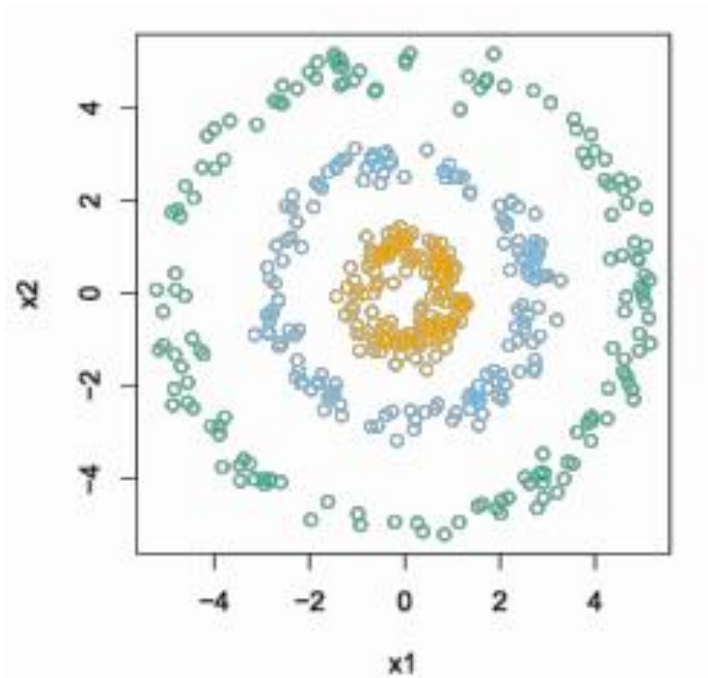
Nonlinear dimensional reduction

- ▶ Goal: Find a low dimensional non-linear map that preserves the relevant information.
 - ▶ Making nonlinear linear
- ▶ Typical methods:
 - ▶ Isomap (Isometric Mapping)
 - ▶ LLE (Locally linear embedding)
 - ▶ KPCA (kernel PCA)
 - ▶ t-SNE (t-distributed stochastic neighbor embedding)
 - ▶ VAE (variational autoencoders)

PCA on non-linear data

Example: Shells

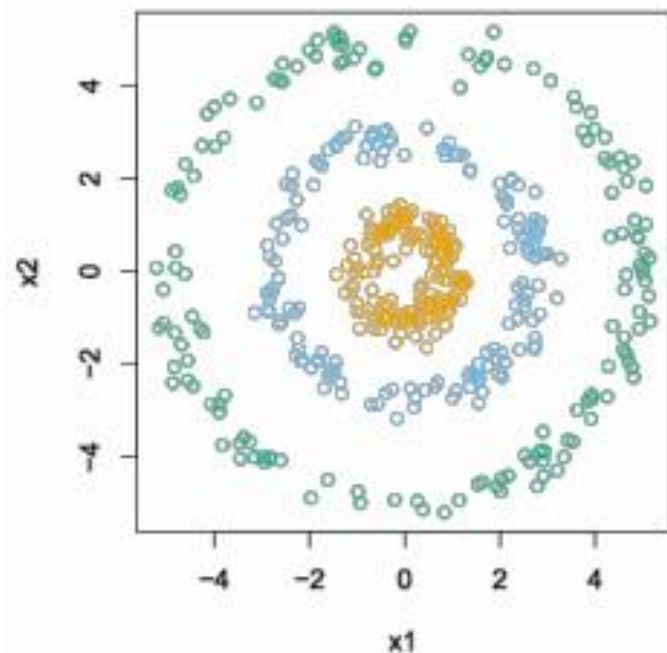
Data



PCA on non-linear data

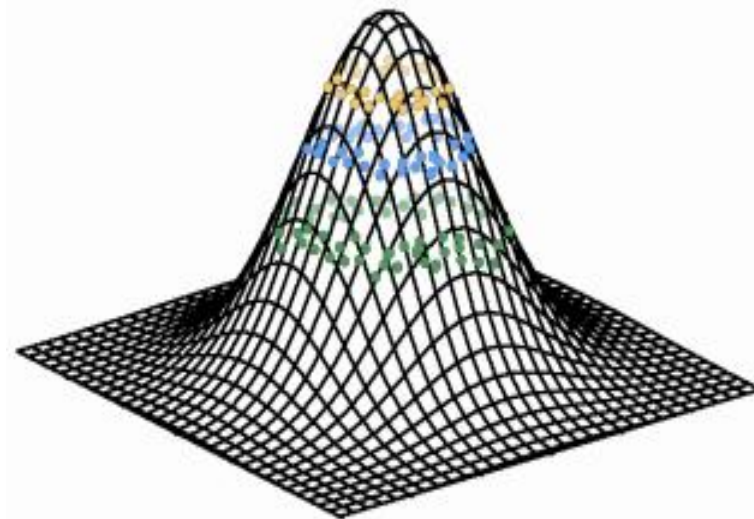
Example: Shells

Data



Feature transformation

- ▶ Key idea: transform inputs x using a feature map $\varphi(x)$
- ▶ $x \in \mathbb{R}^D, \varphi(x) \in \mathbb{R}^C, C > D$



Kernel PCA

PCA

- ▶ $X = [x_1, x_2, \dots, x_N]$
- ▶ $C = \frac{1}{N-1} X X^T$
- ▶ $Cv = \lambda v$
- ▶ Mapping feature: $y = Xv$

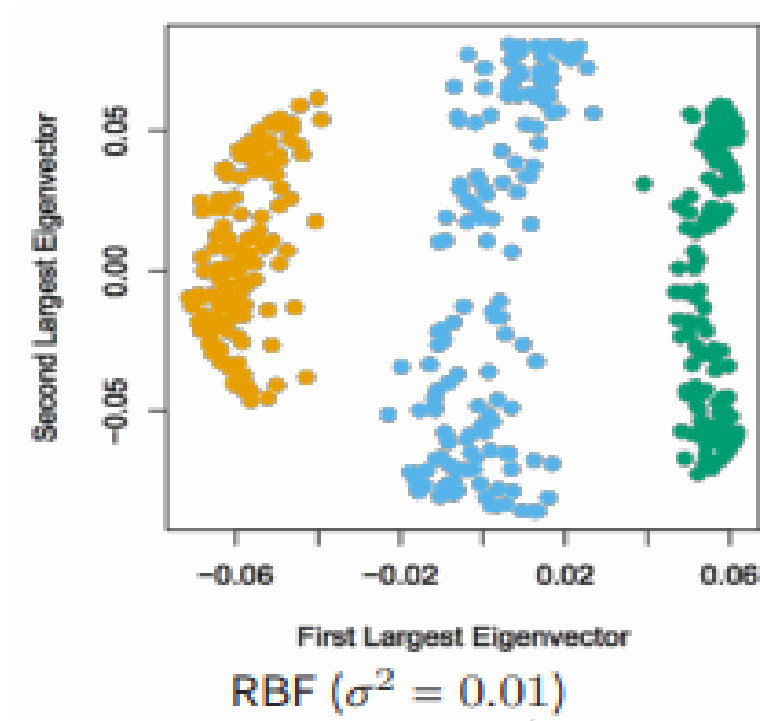
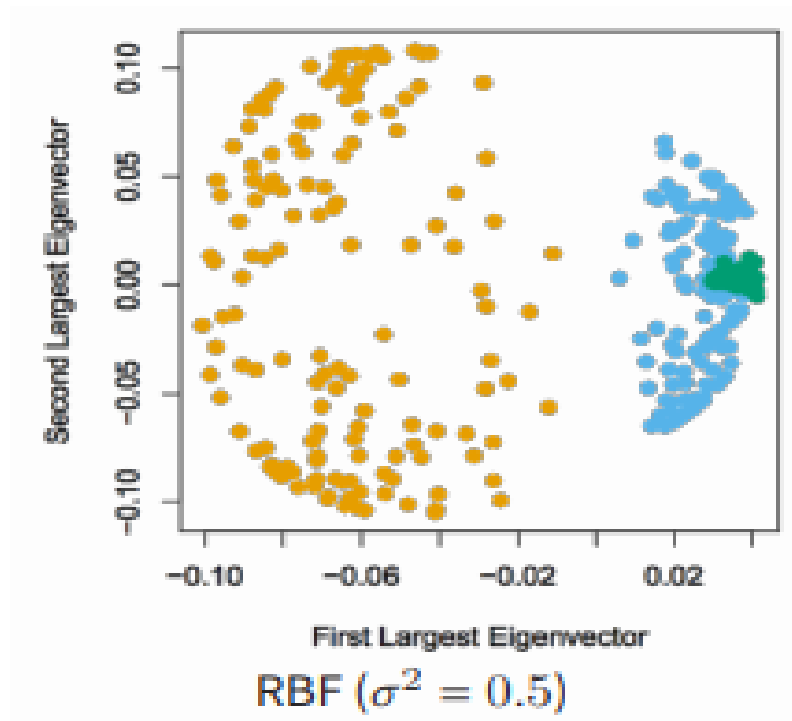
Kernel PCA

- ▶ $X = [x_1, x_2, \dots, x_N]$
- ▶ $\kappa(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$ (kernel function)
- ▶ $\boldsymbol{\varphi}(X) = [\varphi(x_1), \varphi(x_2), \dots, \varphi(x_N)]$
- ▶ $\mathbf{K} = \boldsymbol{\varphi}(X)^T \boldsymbol{\varphi}(X)$ (kernel matrix)
- ▶ $C = \frac{1}{N-1} \boldsymbol{\varphi}(X)^T \boldsymbol{\varphi}(X)$
- ▶ $C\mathbf{a} = \lambda \mathbf{a}$
- ▶ Mapping feature: $y = \boldsymbol{\varphi}(X)\mathbf{a} = \sum_{i=1}^N a_i \varphi(x_i)$

Kernel PCA: Example

- RBF kernel:

- $\kappa(x_i, x_j) = \exp\left(\frac{-|x_i - x_j|^2}{2\sigma^2}\right)$



Kernel PCA

- ▶ Choosing the right kernel
 - ▶ Not an easy choice
 - ▶ Construct by hand / eye where feasible
 - ▶ Possible to learn kernel matrices K directly from data.
- ▶ Some non-linear dimensional reduction methods can be viewed as kernel PCA, with kernels learned from data.
- ▶ Implementation
 - ▶ https://colab.research.google.com/github/nanjakorewa/kdm-notebooks/blob/main/basic/dimensionality_reduction/Kernel-PCA.ipynb#scrollTo=21d37913

Kernel PCA

► Typical kernel

- Polynomial: $\kappa(x_i, x_j) = (< x_i, x_j > + a)^r$, a and r are tuning parameters
- Sigmoid: $\kappa(x_i, x_j) = \tanh(a < x_i, x_j > + b)$, a and b are tuning parameters, neural networks
- Gaussian: $\kappa(x_i, x_j) = \exp(\frac{-|x_i - x_j|^2}{2\sigma^2})$, σ^2 is tuning parameters; radial basis functions

```
from sklearn.decomposition import KernelPCA

kpca = KernelPCA(kernel="rbf", degree=2, gamma=3)
X_kpca = kpca.fit_transform(X)

fig = plt.figure(figsize=(8, 8))
plt.scatter(X_kpca[:, 0], X_kpca[:, 1], c=y)
```

The Polynomial Kernel

- ▶ $K(x_1, x_2) = \langle x_1, x_2 \rangle^2$
 - ▶ $x_1 = (x_{11}, x_{12})$
 - ▶ $x_2 = (x_{21}, x_{22})$
- ▶ $\langle x_1, x_2 \rangle = (x_{11}x_{21} + x_{12}x_{22})$
- ▶ $\langle x_1, x_2 \rangle^2 = (x_{11}^2 x_{21}^2 + x_{12}^2 x_{22}^2 + 2x_{11} x_{12} x_{21} x_{22})$
- ▶ $\Phi(x_1) = (x_{11}^2, x_{12}^2, \sqrt{2}x_{11} x_{12})$
- ▶ $\Phi(x_2) = (x_{21}^2, x_{22}^2, \sqrt{2}x_{21} x_{22})$
- ▶ $K(x_1, x_2) = \langle \Phi(x_1), \Phi(x_2) \rangle$

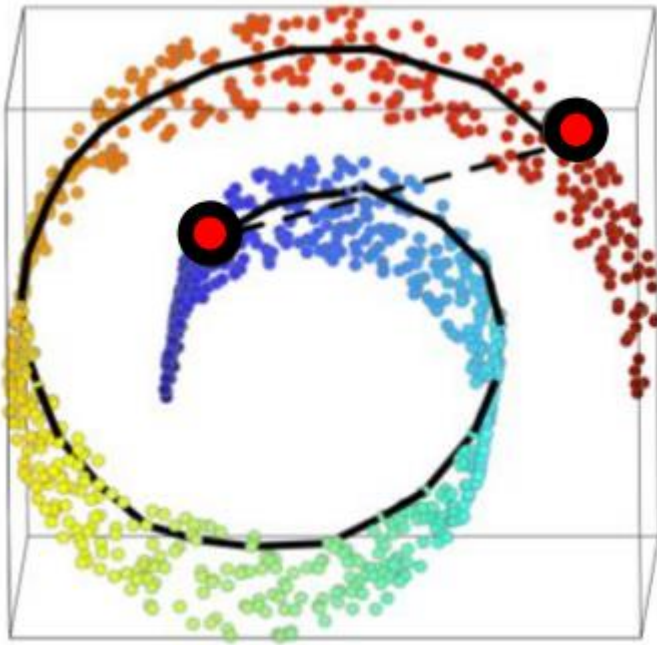
Isomap (Isometric mapping)

- Idea:

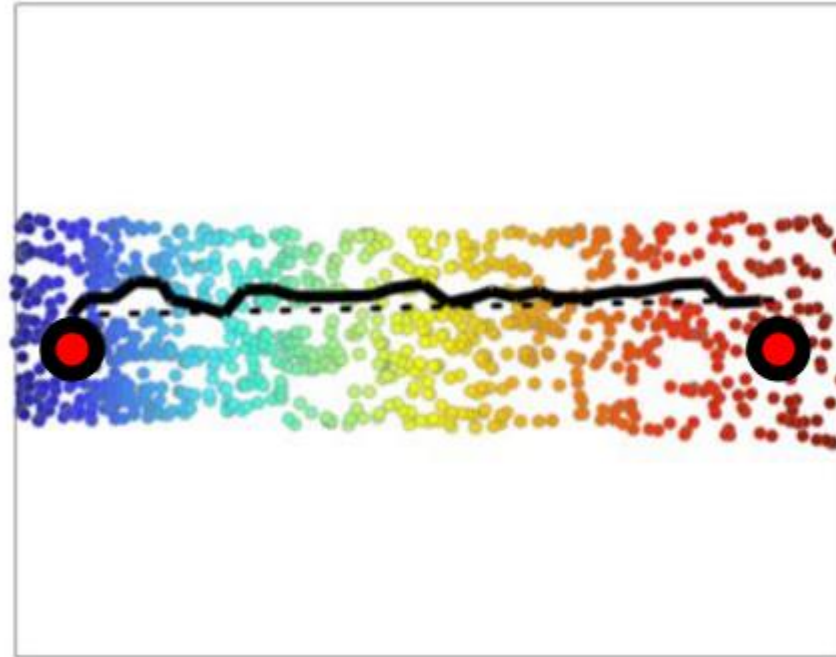
- If geodesics are known, we can use methods like PCA for data embedding.
- Key question: how to estimate the geodesics from data samples.
- Solution: create a k-nearest neighbor graph, and compute (all pairs) shortest paths between data samples.



Isomap (Isometric mapping)



An estimated geodesic via shortest path



Embedding in the lowdimensional space via MDS (which preserves the input geodesic distances)

t-distributed Stochastic Neighbor Embedding (t-SNE)

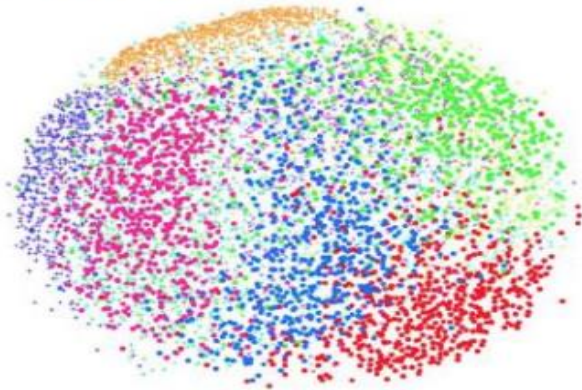
PCA



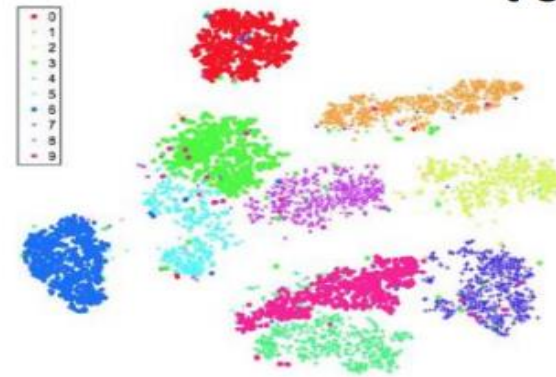
ISOMAP



Sammon mapping



t-SNE



Codes example: t-SNE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import manifold, datasets

digits = datasets.load_digits(n_class=6)
X, y = digits.data, digits.target
n_samples, n_features = X.shape
```

```
1  '''t-SNE'''
2  tsne = manifold.TSNE(n_components=2, init='pca', random_state=501)
3  X_tsne = tsne.fit_transform(X)
4
5  print("Org data dimension is {}.
6        Embedded data dimension is {}".format(X.shape[-1], X_tsne.shape[-1]))
7
8  '''嵌入空间可视化'''
9  x_min, x_max = X_tsne.min(0), X_tsne.max(0)
10 X_norm = (X_tsne - x_min) / (x_max - x_min)  # 归一化
11 plt.figure(figsize=(8, 8))
12 for i in range(X_norm.shape[0]):
13     plt.text(X_norm[i, 0], X_norm[i, 1], str(y[i]), color=plt.cm.Set1(y[i]),
14             fontdict={'weight': 'bold', 'size': 9})
15 plt.xticks([])
16 plt.yticks([])
17 plt.show()
```


Manifold hypothesis

- ▶ High-dimensional data in the real world really lies on low-dimensional manifolds within that high-dimensional space.

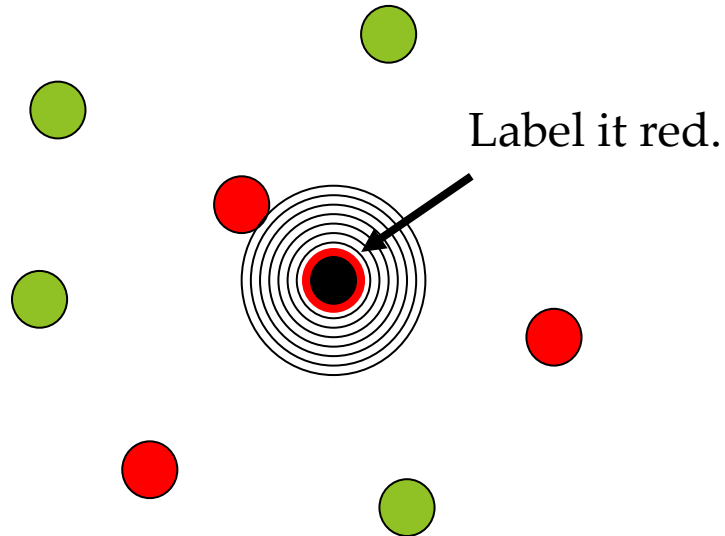
Nonlinear dimensional reduction characteristics

- ▶ Non-linear dimensionality reduction helps visualize complex data in low dimensions—relies on the manifold hypothesis
- ▶ Most methods exploit the nearest neighbor graph in some form (e.g. MDS, Isomap, LLE, etc.)
- ▶ Data is typically required to be clean (not much noise) and dense
- ▶ Many approaches to choose from—t-SNE and UMAP (Uniform Manifold Approximation and Projection) most popular

K-nearest neighbors (KNN)

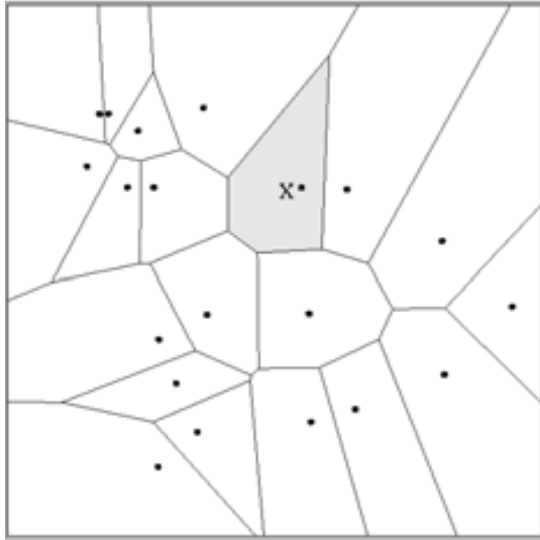
1-Nearest Neighbor

- ▶ One of the simplest of all machine learning classifiers
- ▶ Simple idea: label a new point the same as the closest known point

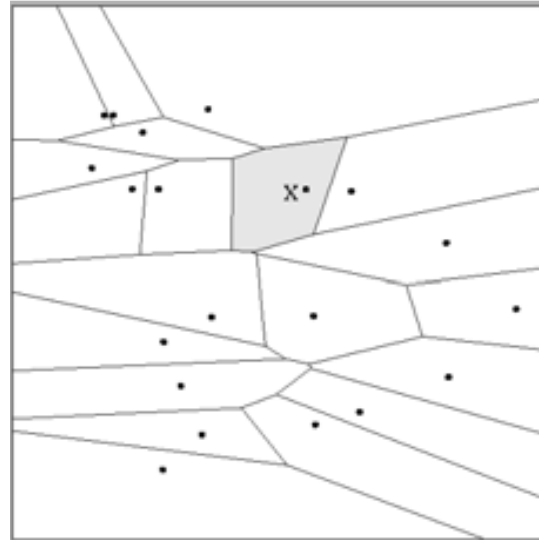


Distance Metrics

- Different metrics can change the decision surface



$$\text{Dist}(\mathbf{a}, \mathbf{b}) = (a_1 - b_1)^2 + (a_2 - b_2)^2$$



$$\text{Dist}(\mathbf{a}, \mathbf{b}) = (a_1 - b_1)^2 + (3a_2 - 3b_2)^2$$

- Standard Euclidean distance metric:
 - Two-dimensional: $\text{Dist}(\mathbf{a}, \mathbf{b}) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$
 - Multivariate: $\text{Dist}(\mathbf{a}, \mathbf{b}) = \sqrt{\sum (a_i - b_i)^2}$

1-NN's Four Aspects as an Instance-Based Learner:

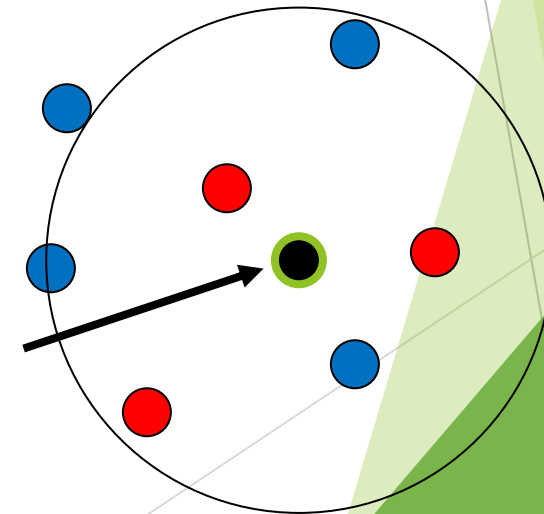
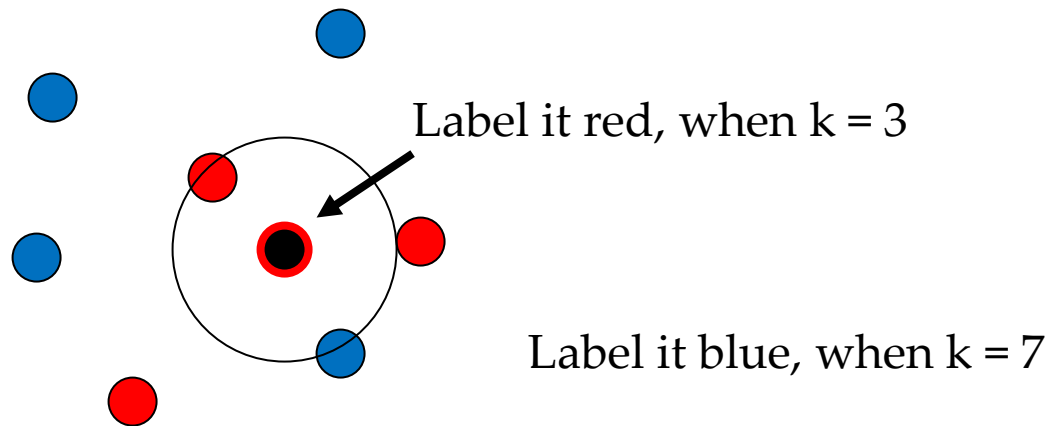
1. A distance metric
2. How many nearby neighbors to look at?
3. A weighting function (optional)
4. How to fit with the local points?

1-NN's Four Aspects as an Instance-Based Learner:

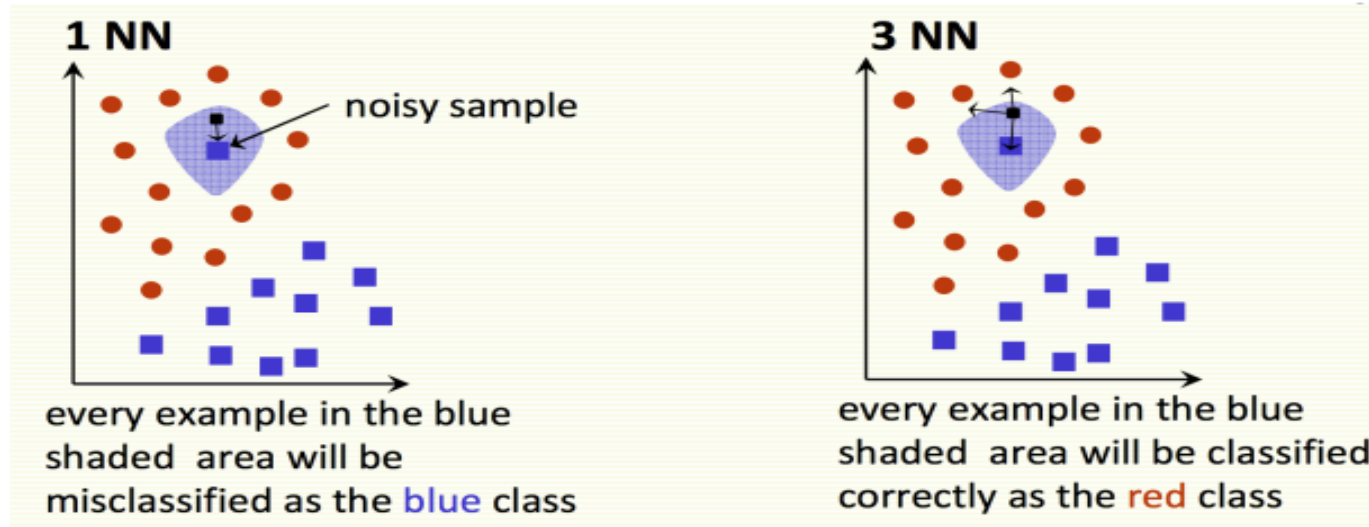
1. A distance metric
 - ▶ *Euclidian*
2. How many nearby neighbors to look at?
 - ▶ *One*
3. A weighting function (optional)
 - ▶ *Unused*
4. How to fit with the local points?
 - ▶ *Just predict the same output as the nearest neighbor.*

k - Nearest Neighbor

- ▶ Generalizes 1-NN to smooth away noise in the labels
- ▶ A new point is now assigned the most frequent label of its k nearest neighbors



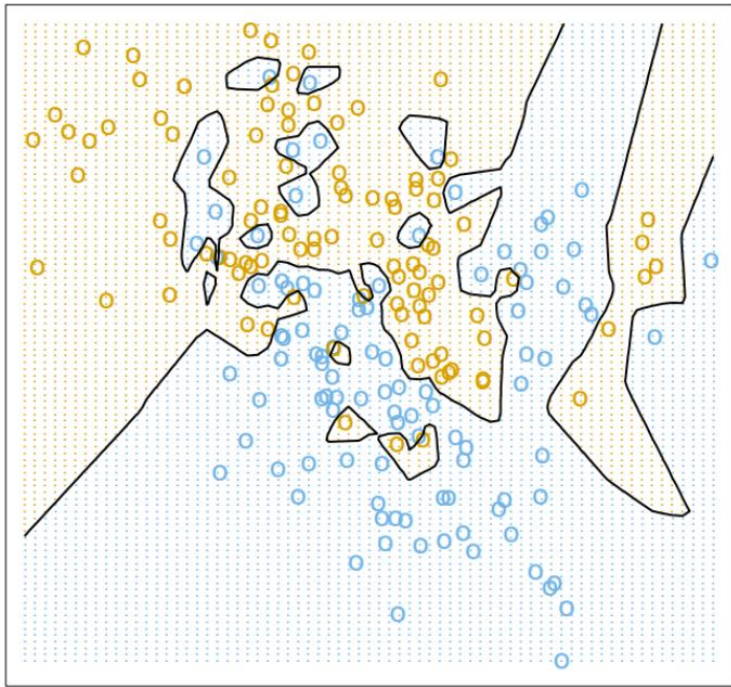
k - Nearest Neighbor



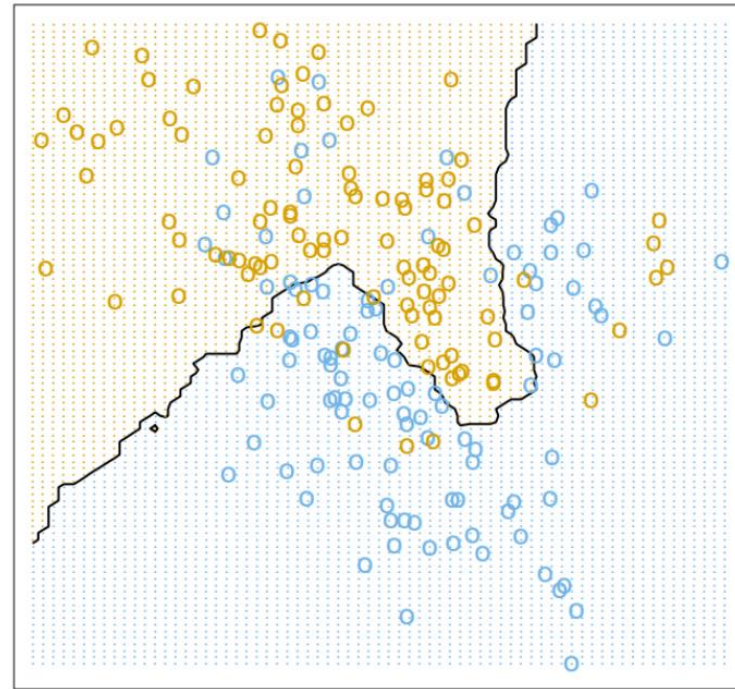
- ▶ Nearest neighbors **sensitive to noise** or **mis-labeled data** (“class noise”). Solution?
- ▶ Smooth by having K nearest neighbors' vote

k - Nearest Neighbor

K=1



K=2



Codes

```
# Applying K nearest Neighbour algorithm to predict the results
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

clf1=KNeighborsClassifier(n_neighbors=3)
#clf2=clf1
clf1.fit(X_pca_train, y_train)
#clf1.fit(X_train, y_train)
print(X_pca_train.shape)
y_pred_pca=clf1.predict(X_pca_test)
#y_pred=clf1.predict(X_test)
```

```
cf_matrix_pca = confusion_matrix(y_test, y_pred_pca)
sns.set(rc = {'figure.figsize':(8,3)})
#sns.heatmap(cf_matrix, annot=True)

sns.heatmap(cf_matrix_pca, annot=True)
print(classification_report(y_test, y_pred_pca))
```

Support vector machine (SVM)

- Prediction
 - Why might predictions be wrong?
- Support vector machines
 - Doing really well with linear models
- Kernels
 - Making the non-linear linear

Supervised learning = Prediction

- ▶ Given training instances (x,y)
- ▶ Learn a model f
- ▶ Such that $f(x) = y$
- ▶ Use f to predict y for new x
- ▶ Many variations on this basic theme

Why might predictions be wrong?

- ▶ True Non-Determinism
 - ▶ Flip a biased coin
 - ▶ $p(\text{heads}) = \theta$
 - ▶ Estimate θ
 - ▶ If $\theta > 0.5$ predict `heads`, **else** `tails`
 - ▶ Lots of ML research on problems like this
 - ▶ Learn a model
 - ▶ Do the best you can in expectation

Why might predictions be wrong?

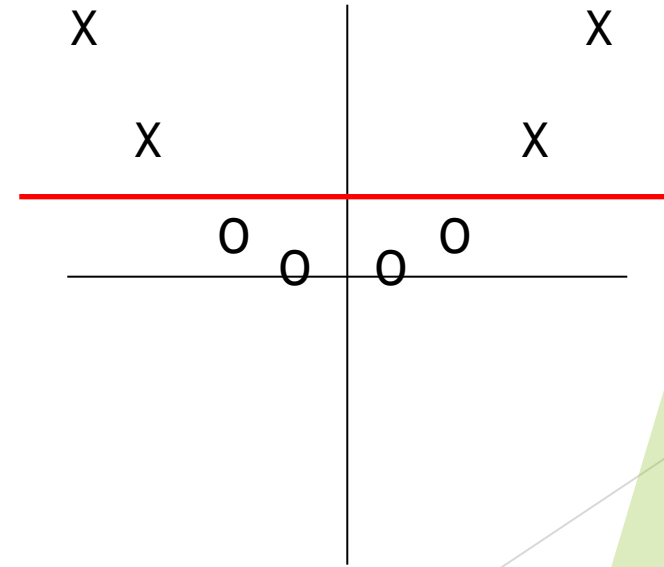
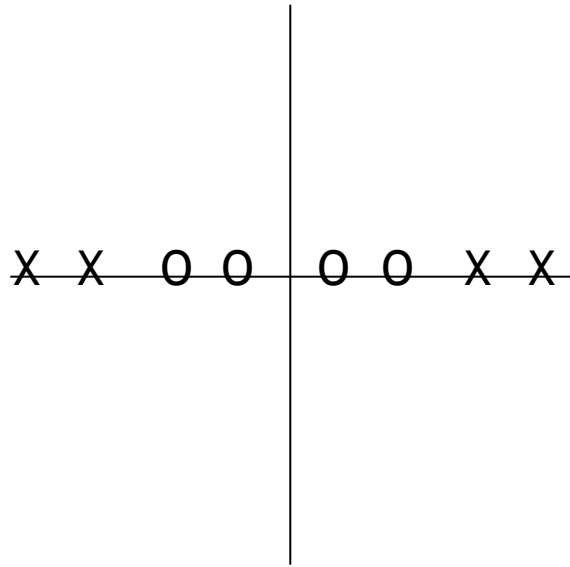
- ▶ Partial Observability (PO)
 - ▶ Something needed to predict y is missing from observation x
- ▶ N -bit parity problem
 - ▶ x contains $N-1$ bits (hard PO)
 - ▶ x contains N bits, but learner ignores some of them (soft PO)

Why might predictions be wrong?

- ▶ True non-determinism
- ▶ Partial observability
 - ▶ hard, soft
- ▶ Representational bias
- ▶ Algorithmic bias
- ▶ Bounded resources

Representational Bias

- Having the right features (x) is crucial



Strengths of SVMs

- ▶ Good generalization in theory
- ▶ Good generalization in practice
- ▶ Work well with few training instances
- ▶ Find globally best model
- ▶ Efficient algorithms
- ▶ Amenable to the kernel trick

Linear Separators

- ▶ Training instances
 - ▶ $x \in \mathbb{R}^n$
 - ▶ $y \in \{-1, 1\}$
- ▶ $w \in \mathbb{R}^n$
- ▶ $b \in \mathbb{R}$
- ▶ Hyperplane
 - ▶ $\langle w, x \rangle + b = 0$
 - ▶ $w_1x_1 + w_2x_2 \dots + w_nx_n + b = 0$
- ▶ Decision function
 - ▶ $f(x) = \text{sign}(\langle w, x \rangle + b)$

Math Review

Inner (dot) product:

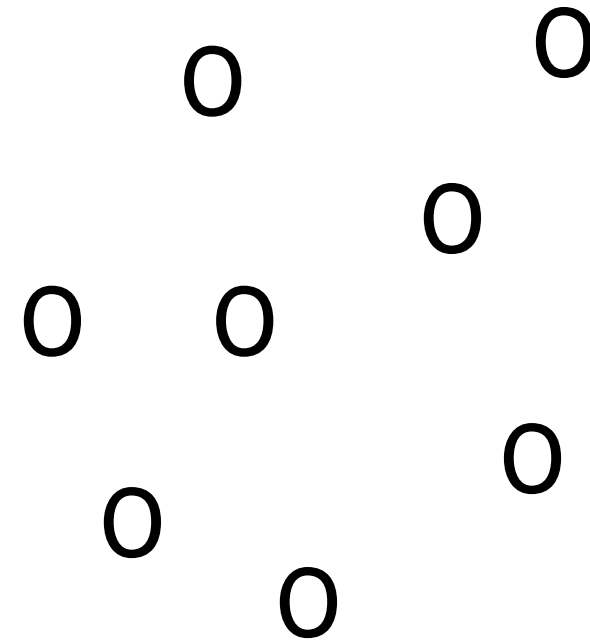
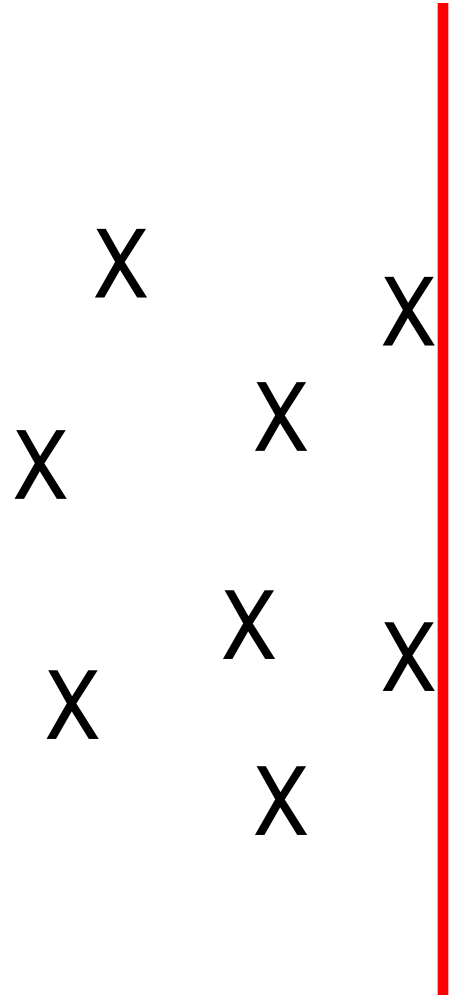
$$\begin{aligned}\langle a, b \rangle &= a \cdot b = \sum a_i * b_i \\ &= a_1b_1 + a_2b_2 + \dots + a_nb_n\end{aligned}$$

Intuitions

X
X X X
X X X
X X

0 0
0 0 0
0 0 0

Intuitions

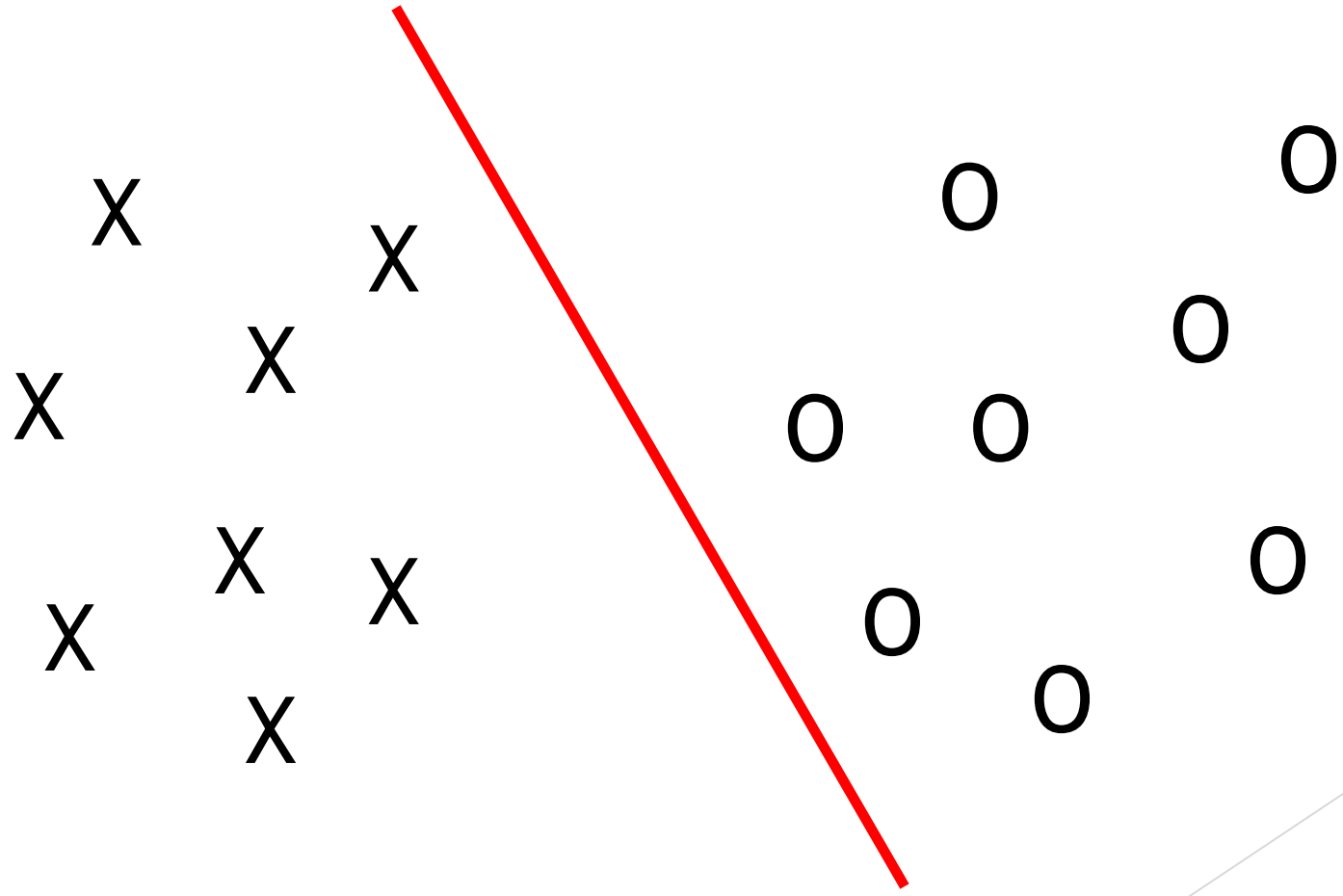


Intuitions

X
X X X
X X X
X X

0 0 0
0 0 0
0 0 0

Intuitions

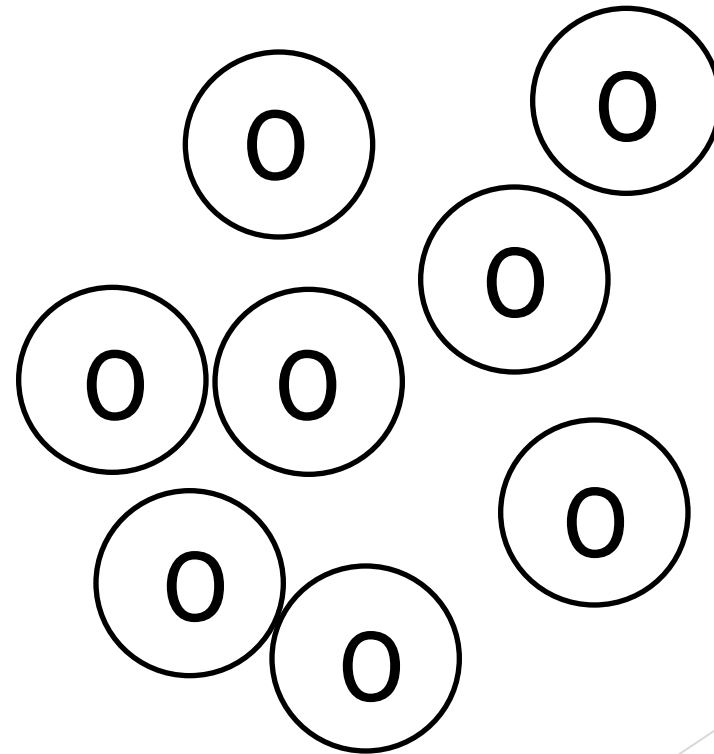
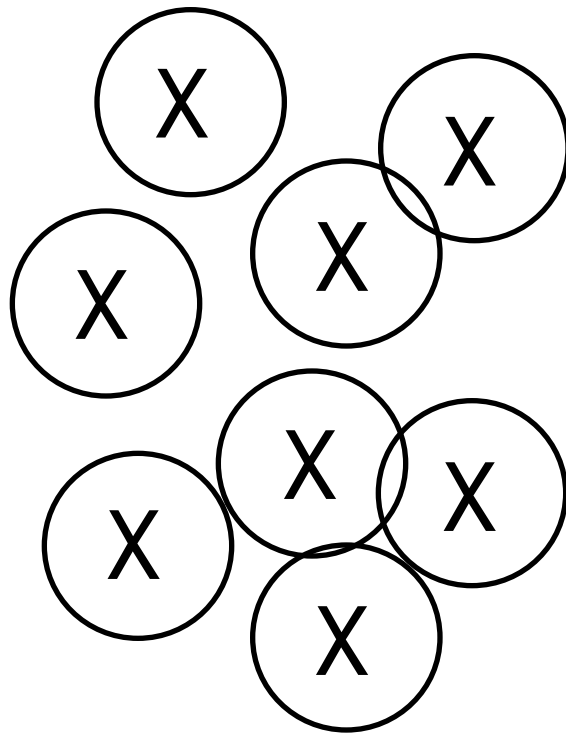


A “Good” Separator

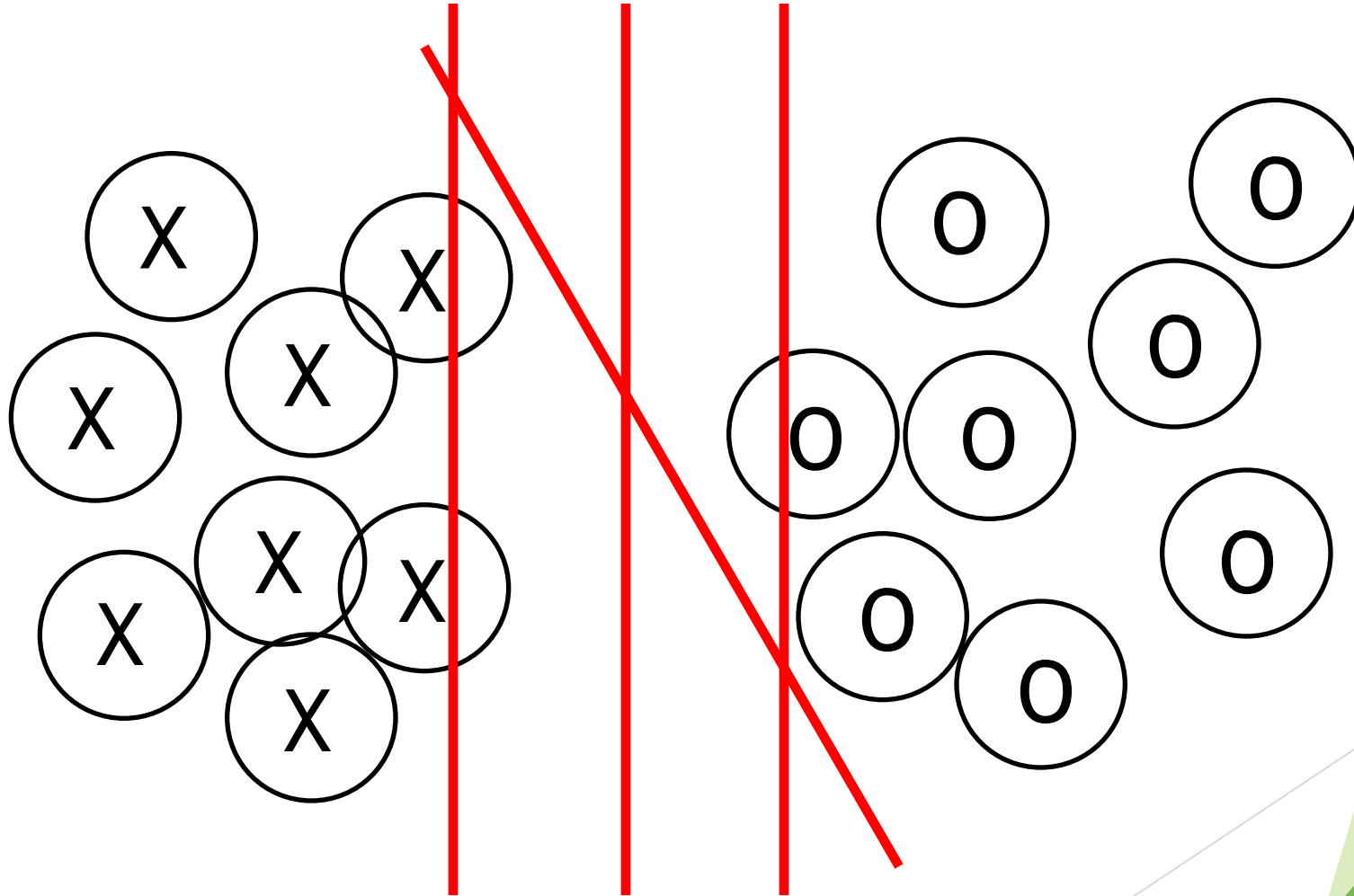
X X X
X X
X X X
X

0 0
0 0 0
0 0 0

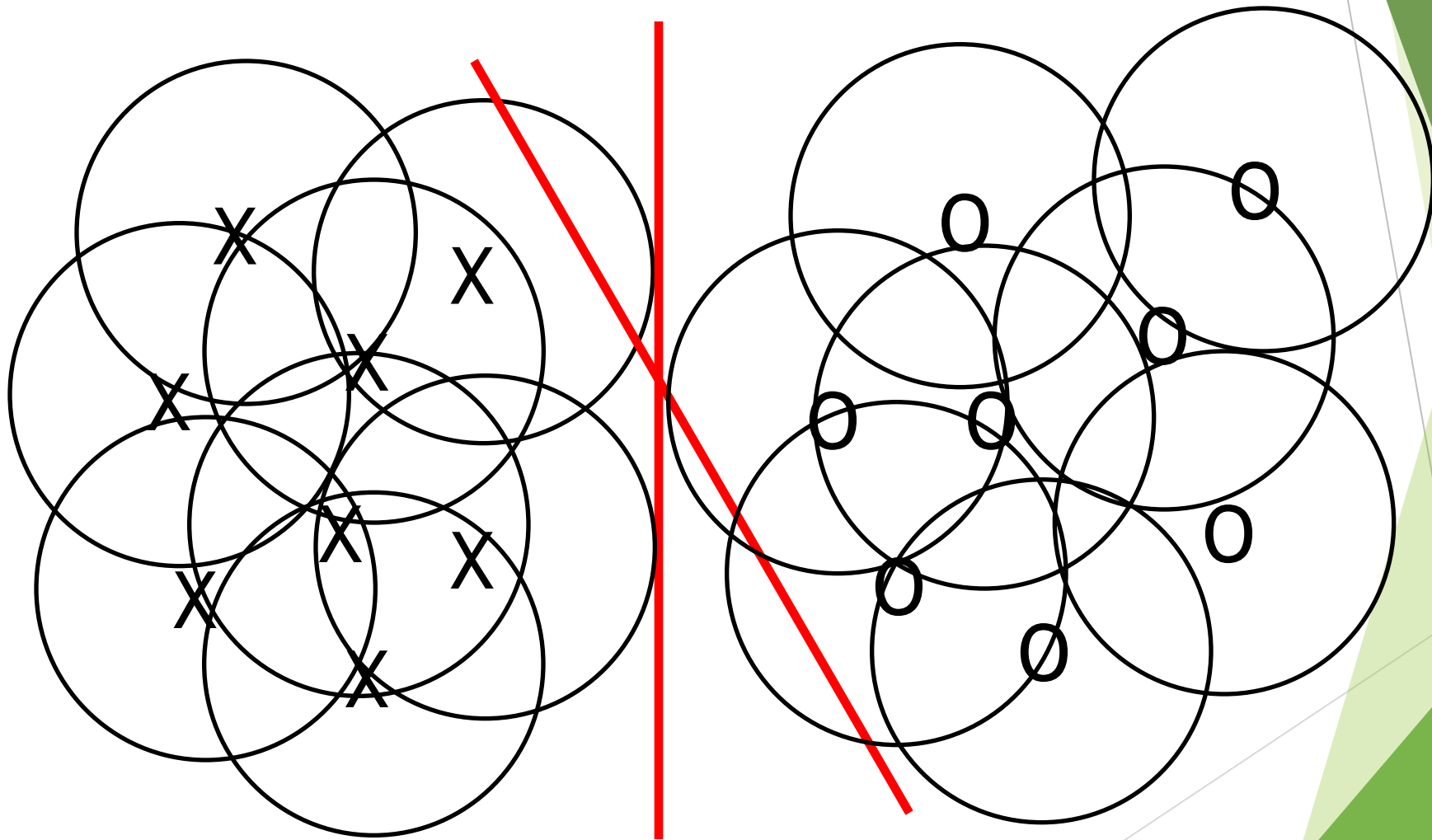
Noise in the Observations



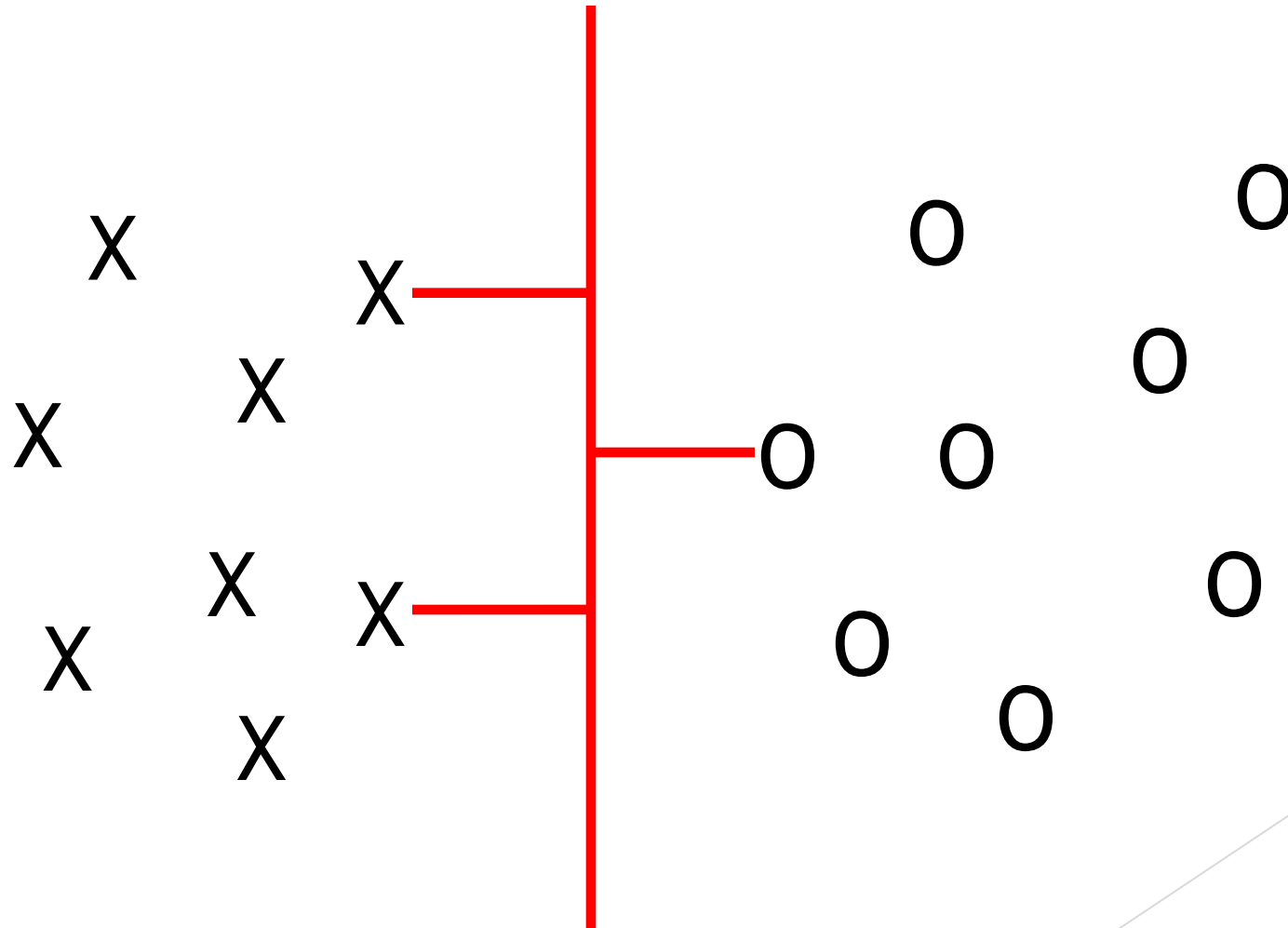
Ruling Out Some Separators



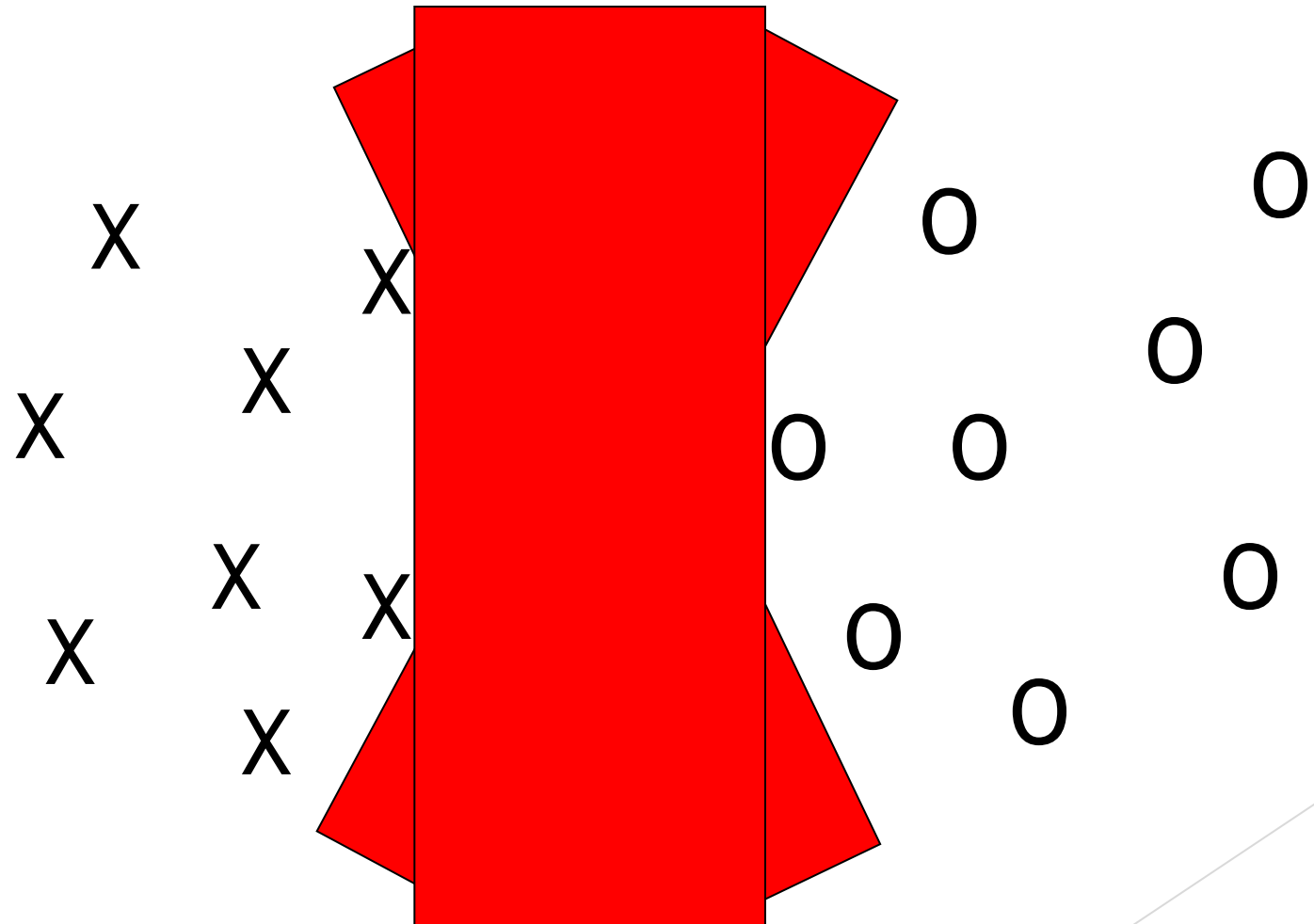
Lots of Noise



Maximizing the Margin



“Fat” Separators



The Math

- ▶ Training instances
 - ▶ $x \in \mathbb{R}^n$
 - ▶ $y \in \{-1, 1\}$
- ▶ Decision function
 - ▶ $f(x) = \text{sign}(\langle w, x \rangle + b)$
 - ▶ $w \in \mathbb{R}^n$
 - ▶ $b \in \mathbb{R}$
- ▶ Find w and b that
 - ▶ Perfectly classify training instances
 - ▶ Assuming linear separability
 - ▶ Maximize margin

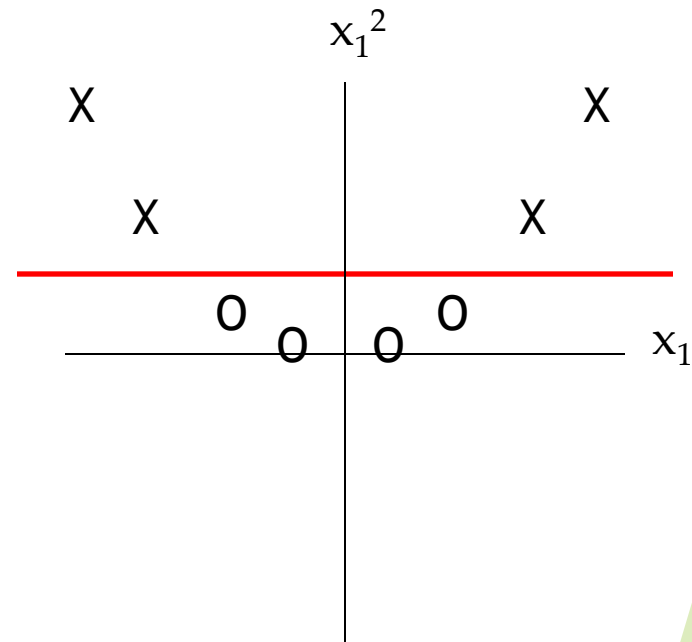
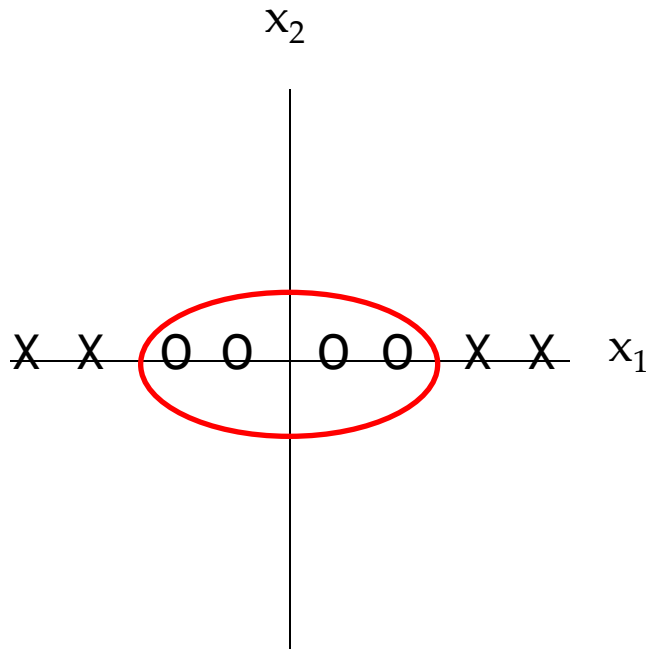
The Math

- ▶ For perfect classification, we want
 - ▶ $y_i (\langle w, x_i \rangle + b) \geq 0$ for all i
- ▶ To maximize the margin, we want
 - ▶ w that minimizes $|w|^2$

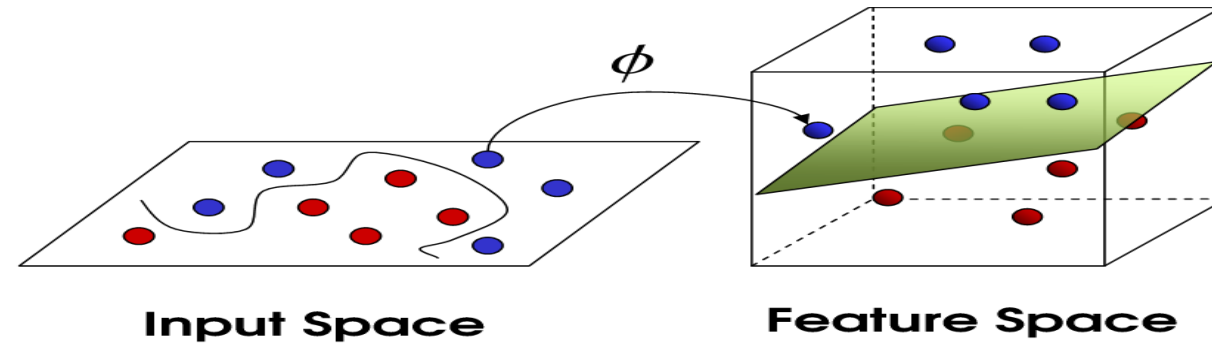
Dual Optimization Problem

- ▶ Maximize over α
 - ▶ $W(\alpha) = \sum_i \alpha_i - 1/2 \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$
- ▶ Subject to
 - ▶ $\alpha_i \geq 0$
 - ▶ $\sum_i \alpha_i y_i = 0$
- ▶ Decision function
 - ▶ $f(x) = \text{sign}(\sum_i \alpha_i y_i \langle x, x_i \rangle + b)$

When Linear Separators Fail



Mapping into a New Feature Space



$$\Phi : x \rightarrow X = \Phi(x)$$

$$\Phi(x_1, x_2) = (x_1, x_2, x_1^2, x_2^2, x_1x_2)$$

- ▶ Rather than run SVM on x_i , run it on $\Phi(x_i)$
- ▶ Find non-linear separator in input space
- ▶ What if $\Phi(x_i)$ is really big?
- ▶ Use kernels to compute it implicitly!

Kernels

- ▶ Find kernel K such that
 - ▶ $K(x_1, x_2) = \langle \Phi(x_1), \Phi(x_2) \rangle$
- ▶ Computing $K(x_1, x_2)$ should be efficient
- ▶ Use $K(x_1, x_2)$ in SVM algorithm rather than $\langle x_1, x_2 \rangle$